

vla.simd: Efficient CPU Inference for Language-Conditioned Manipulation

Anonymous Authors

Abstract—Deploying language-conditioned manipulation policies without a dedicated GPU requires both efficient inference and a policy that can supply actions within the controller’s timing budget. We study when CPUs meet this requirement by relating action-chunking latency and execution horizon to the supply of usable actions. We present `vla.simd`, a CPU inference engine that combines shared SIMD micro-kernels, reusable computation, and target-specific implementation choices. We also introduce IMPACT, an ACT-based policy that adds language conditioning through cached text representations and visual modulation. Across six policies and four CPUs, `vla.simd` achieves a median speedup of approximately $1.4\times$ over compiled PyTorch references while maintaining numerical fidelity. The gains can move policies across the control budget, but the best implementation choices depend on the processor. IMPACT is the only language-conditioned policy in our evaluated set that supplies at least 30 actions per second on the Raspberry Pi 5, including after a thermal soak. It achieves 76.4% success on LIBERO without robot pretraining, and instruction-shuffling tests show that its behavior follows the supplied goal. Unlike ACT, which lacks language conditioning, IMPACT uses the instruction to select the task, as demonstrated by destination selection in a shared real-robot scene. Real-robot trials cover two embodiments: an SO-101 arm running IMPACT, and a UR10e with a Robotiq gripper running SmolVLA under `vla.simd`. Project page: <https://vla-simd.github.io/>.

I. INTRODUCTION

Deploying a language-conditioned manipulation policy [1], [2], [3], [4] on a robot without a dedicated GPU raises a practical question: can the available CPU deliver actions fast enough to keep the robot moving? Answering this requires more than comparing model sizes or query latencies. A manipulation policy must provide actions within the timing constraints of its controller while retaining the ability to select the goal specified by the user.

Action chunking creates an opportunity for CPU deployment. By predicting several future actions in each query [5], [6], a policy can overlap inference with execution and operate below the controller’s update frequency [7], [4]. The relevant constraint is therefore whether the next usable action chunk arrives before the current one is exhausted. This depends jointly on inference latency, execution horizon, and how delayed actions are handled (Section III, Fig. 1). A faster query does not necessarily provide a higher action-supply rate, and meeting a nominal compute budget does not by itself guarantee responsive or successful manipulation. For multi-task deployment, two challenges remain: fitting execution within this budget on the target CPU and preserving language-based goal selection without exceeding it.

We investigate how CPU implementation can expand the

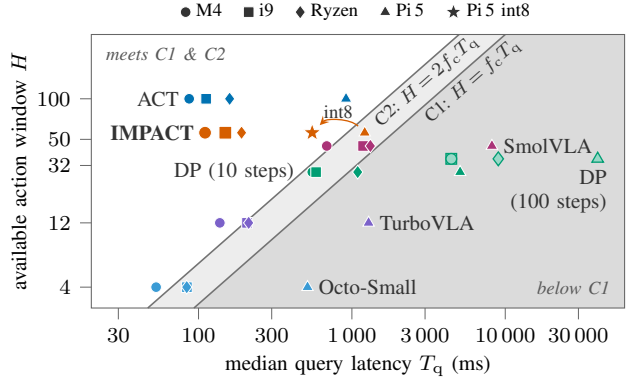


Fig. 1. Real-time budget of chunked policies. Each marker is one policy on one CPU, at the policy’s chunk length H and its median query latency T_q ; the two rows that share each H are drawn a factor of 1.12 above and below it, which moves no marker across a line. Above C1 a policy supplies actions at least as fast as a $f_c = 30$ Hz controller consumes them; above C2 it still does when stale actions are discarded (Section III). On the Pi 5, ACT and TurboVLA differ by $1.4\times$ in latency but execute 100 and 12 actions per query, so ACT meets C1 and TurboVLA does not; IMPACT meets C1 in fp32 and C2 with int8.

set of policies that meet this budget. Linear layers, convolutions, and attention projections share matrix computations that can exploit SIMD instructions through packed-panel micro-kernels [8], [9]. Data layout, register use, memory reuse, and threading must suit the policy and processor: a setting that helps one CPU can hurt another. Our engine, `vla.simd`, combines shared SIMD micro-kernels with target-specific choices and separates *computation lifetimes*: weights are packed per process, instruction-only tensors are cached per episode, and observation-dependent features are computed per query.

Efficient execution must also preserve the information needed to choose the task. ACT [5] provides an efficient starting point, meeting the basic 30 Hz action-supply budget on the Pi 5, but it does not condition on language instructions. In a multi-task setting, the same scene can correspond to different goals, such as placing the tape into either the box or the cup; the observation alone cannot specify the user’s intention. We therefore introduce IMPACT (Instruction-Modulated Perception and Action Chunking with Transformers), an action-chunking policy that extends ACT with language conditioning through cached text representations, instruction tokens, and visual modulation. We evaluate speed, numerical fidelity, and action-supply budgets across four CPUs, alongside manipulation in simulation and on two robots. The results show that CPU optimization can move existing policies across the

nominal control budget, while IMPACT is the only language-conditioned policy in our evaluated set that supplies at least 30 actions per second on the Raspberry Pi 5.

Our contributions are:

- **vla.simd**, a CPU inference engine that shares a SIMD micro-kernel across linear, convolution and attention-projection layers, with target-specific register tiles, runtime settings, and quantization.
- **IMPACT**, a language-conditioned action-chunking policy with a language-conditioning cost of at most 2.6% of per-query compute, evaluated through LIBERO, instruction shuffling and real SO-101 manipulation.
- **A study of six policies on four CPUs and two embodiments**, covering sustained action supply, PyTorch fidelity, and robot deployment. `vla.simd` moves Octo-Small and SmolVLA across 30 Hz on the Ryzen; IMPACT alone among evaluated language-conditioned policies meets this budget on the Pi 5, and its int8 path meets the stricter time-aligned budget.

II. RELATED WORK

Efficient vision-language-action policies. Billion-parameter vision-language-action (VLA) models [1], [2], [10] motivated compact designs that shorten or remove the language model [4], [11], [12], reduce the overall parameter count [13], [14], or quantize the policy [15]; see [16] for a survey. Recent studies characterize VLA latency on GPUs and embedded accelerators [17], [18], with broader hardware comparisons also including CPUs [19]. Lite VLA reports CPU deployment on a Pi 4, with inference taking seconds to minutes [20]. We compare six policies on four CPUs using the same timing boundary and account for the actions supplied by each query.

Chunked execution in real time. ACT [5] and Diffusion Policy [6] predict action chunks. Real-time chunking [7] overlaps inference with motion and guides new chunks for continuity; LeRobot supports asynchronous serving [4], [21]. Bidirectional decoding [22] addresses the consistency-reactivity trade-off through guided sampling. We apply the action-availability condition of [7] to sequential CPU queries and distinguish lagged from time-aligned execution.

Language conditioning. Feature-wise linear modulation (FiLM) [23] uses language to scale and shift visual features in BC-Z [24], RT-1 [25], and BAKU [26]. MT-ACT [27] combines FiLM in the backbone with a language token in an ACT policy, and Octo [3] reads frozen T5 tokens. IMPACT combines these established mechanisms while caching instruction-only features to limit the language-processing cost of each CPU query.

CPU inference. Our micro-kernel follows GotoBLAS and BLIS [8], [9], with register constraints based on Low et al. [28] and empirical validation on each target. Production runtimes such as XNNPACK [29], ONNX Runtime [30], OpenVINO [31], ExecuTorch [32], and llama.cpp [33] select kernels for each instruction set architecture (ISA). `vla.cpp` [34] extends llama.cpp with cached multimodal prefixes and iterative action heads; its reported evaluation

uses discrete and embedded GPUs. SIMD also accelerates motion planning on CPUs [35], [36]. `vla.simd` focuses on CPU execution across policy families, testing how kernel, threading, and precision choices affect action-supply budgets.

III. ACTION-SUPPLY BUDGETS

We first determine when a policy can supply actions continuously despite inference delay. A controller running at frequency f_c consumes one action every $\Delta = 1/f_c$ seconds. A policy predicts N actions from the latest observation, of which a window of $H \leq N$ actions is available for execution. For example, our Diffusion Policy predicts 64 actions but exposes a 32-action window; the other settings use their full predicted chunk (Table II). We index this window from the observation’s control step t and denote query latency by T_q , or $d = \lceil f_c T_q \rceil$ control steps. The following analysis assumes fixed delay, steady-state execution, and sequential queries starting $s \geq d$ steps apart. The ratio $f_{\text{eff}} = H/T_q$, termed effective control rate in [34], is an *action-supply rate*; it does not measure feedback frequency.

We consider two execution modes. In *lagged* execution, the robot starts each chunk from its first action upon arrival. To avoid running out of actions, the next chunk must arrive before the current one is exhausted, requiring $s \leq H$. In *time-aligned* execution, actions retain their intended control steps, and stale actions are discarded, as in real-time chunking [7] and the LeRobot client for our robot [4], [21]. Only $H - d$ actions remain when a chunk arrives, so covering the next arrival requires $s + d \leq H$, equivalent to $d \leq H - s$ in [7].

Combining these requirements with $s \geq d$ gives the action-supply conditions

$$(C1) \quad d \leq H, \quad (C2) \quad 2d \leq H, \quad (1)$$

for lagged (C1) and time-aligned (C2) execution, respectively, because the shortest query interval is $s = d$. C1 is equivalent to $f_{\text{eff}} \geq f_c$; ignoring rounding to control steps gives the approximation $f_{\text{eff}} \geq 2f_c$ for C2. Thus, time-aligned execution needs enough actions to cover both the current inference delay and the wait for the next chunk. These conditions ensure action availability under the stated model; smooth chunk transitions and task success require separate evaluation. Each executed action uses an observation at least d steps old.

We evaluate these nominal budgets using median engine latency, excluding camera capture and transport. Reliable deployment must also account for these additional delays and occasional slow queries. We use $f_c = 30$ Hz, matching our SO-101 controller [37] and the frame rate of its training datasets.

IV. THE VLA.SIMD ENGINE

A. Computation Reuse

`vla.simd` maps raw camera frames, joint state, and an instruction to an action chunk in robot units, without a deep learning framework at run time. It handles resizing, normalization, and tokenization; query timing excludes camera capture, transport, and episode initialization.

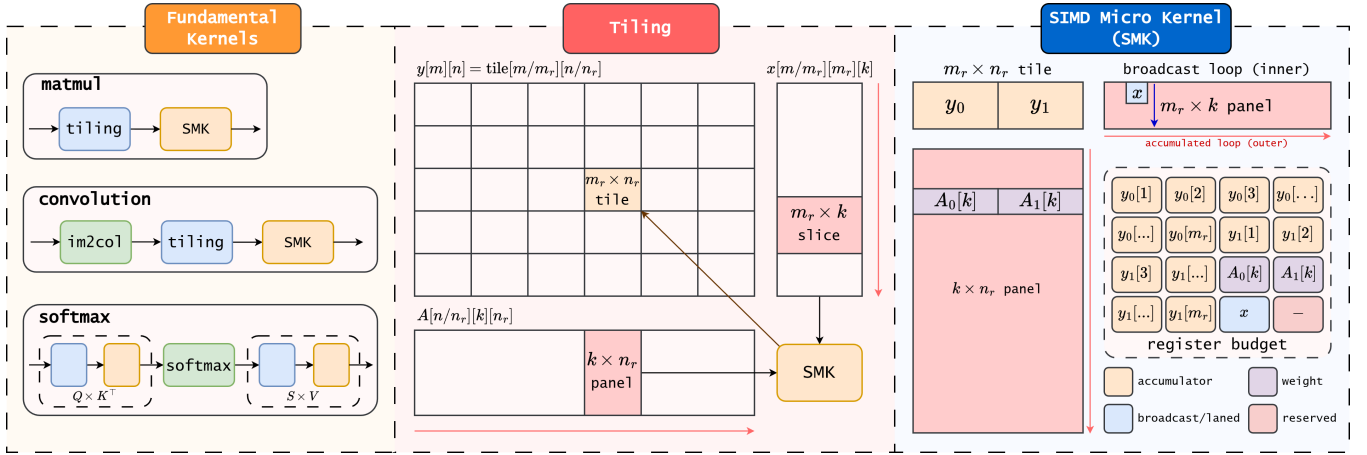


Fig. 2. SIMD execution in `vla.simd`. Linear layers, convolutions, and attention projections share a packed-panel SIMD micro-kernel (SMK). Convolution supplies image patches to this kernel one panel at a time. The $m_r \times n_r$ output tile remains in registers during accumulation (right). The attention products QK^T and $\text{softmax}(\cdot)V$ use a separate per-backend kernel, with K^T written directly by the key projection.

Weights are converted offline to contiguous fp32 arrays and packed once per process; instruction-only tensors are cached per episode. Each query computes observation-dependent features, including image-dependent language features. Shared operators dispatch to Intel AVX2, AMD AVX2, ARMv8 NEON, or Apple silicon backends. Apple uses Accelerate [38] for dense matrix products and convolutions by default; the others use the shared micro-kernel (SMK).

B. Selecting a Register Tile for Each Target

A linear layer computes $Y = XW^T + 1b^T$ for inputs $X \in \mathbb{R}^{m \times k}$, weights $W \in \mathbb{R}^{n \times k}$, and bias $b \in \mathbb{R}^n$. Following GotoBLAS and BLIS [8], [9], weights are packed into panels of n_r output columns. The micro-kernel accumulates an $m_r \times n_r$ output tile in vector registers over depth k , then adds bias on storage. Fused multiply-add (FMA) instructions follow increasing depth order, preserving each output’s accumulation order across tile dimensions.

With L fp32 values per vector register and $n_v = n_r/L$ registers per tile row, a tile requires $m_r n_v$ accumulator registers, n_v weight registers, and q activation registers. AVX2 broadcasts each input value to one register, reusing it for each row, so $q = 1$ and each depth step issues $\Lambda = n_v + m_r$ loads. NEON instead keeps one vector per row and reads its lanes over L consecutive depth steps, giving $q = m_r$ and an average of $\Lambda = n_v + m_r/L$ loads per step for unpacked activations. For a target with R_{max} vector registers, throughputs of P_{fma} vector FMAs and P_{ld} vector loads per cycle, and FMA latency τ cycles, we require

$$m_r n_v + n_v + q \leq R_{\text{max}}, \quad (2)$$

$$m_r n_v \geq P_{\text{fma}} \tau, \quad (3)$$

$$m_r n_v P_{\text{ld}} \geq \Lambda P_{\text{fma}}, \quad (4)$$

Equation (2) limits register use, (3) provides enough independent accumulators to hide FMA latency [28], and (4) requires loads to keep pace with arithmetic. We also inspect the compiled inner loop and reject tiles that spill registers to stack memory.

Algorithm 1 Register tile for one target

Require: $L, R_{\text{max}}, P_{\text{fma}}, P_{\text{ld}}, \tau$, operand mode, widths $\mathcal{N}$ of the packed layers

- 1: $g \leftarrow \text{gcd } \mathcal{N}$; best $\leftarrow \perp$ $\triangleright \perp$: unpacked fallback
- 2: **for** each n_r with $L \mid n_r$ and $n_r \mid g$ **do**
- 3: **for** $m_r \leftarrow R_{\text{max}}$ **down to** 1 **do** $\triangleright \phi$ grows with m_r
- 4: **if** (2)–(4) hold **and not** SPILLS(m_r, n_r) **then**
- 5: **if** best = $\perp$ **or** $\phi(m_r, n_r) > \phi(\text{best})$ **then**
- 6: best $\leftarrow (m_r, n_r)$
- 7: **break** $\triangleright$ next n_r
- 8: **return** best

Algorithm 1 considers widths n_r divisible by L and dividing the packed layers’ greatest common output-width divisor g . For each, it retains the largest feasible m_r and maximizes $\phi = m_r n_v / (n_v + m_r)$, falling back to unpacked weights if no tile qualifies. The score is the AVX2 broadcast kernel’s FMA-to-load ratio; its use for NEON is an empirical heuristic. The NEON load model alone favors 9×8 and 8×8 , yet the selected 4×16 tile is 12% and 19% faster on the Cortex-A76.

For our policy widths ($g = 64$), this procedure selects 6×16 on AVX2, using 15 of 16 registers, and 4×16 on NEON, using 24 of 32. The AVX2 choice matches the single-precision tile used by BLIS for Haswell and Zen. On the Pi 5, the selected tile reaches 106.9 GFLOP/s with four threads and 96% of peak arithmetic throughput on one core with data in L2 cache. On both ISAs, the selected tile is the fastest tested candidate across all six benchmarked layer shapes.

C. Convolution and Attention

Convolution reuses the micro-kernel through `im2col` [39]: rows contain image patches and packed weights contain flattened filters. Constructing patches one panel at a time respects stride and padding without storing the full patch matrix.

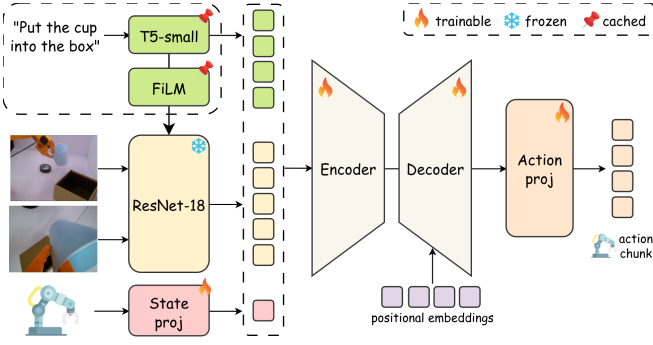


Fig. 3. IMPACT. A frozen T5-small encodes the instruction once per episode, providing transformer tokens and FiLM coefficients for each ResNet-18 stage. Each query runs the modulated backbone and encoder-decoder to predict a 50-step action chunk.

Attention’s query (Q), key (K), value (V), and output projections use the same linear operator; the key projection writes $K^\top$ directly by transposing tiles on storage. The attention products $QK^\top$ and $\text{softmax}(QK^\top/\sqrt{d_h})V$, where d_h is the head width, use a separate per-backend kernel. Both operands change with each query, and the key count need not be a multiple of n_r . Linear layers whose output width is not a multiple of n_r , such as action heads, use a row-blocked kernel.

D. Runtime Settings and Quantization

The backend and register tile are fixed at build time. Runtime settings control weight packing, convolution panels, matrix-multiplication blocking, attention kernels, and threading (Table IV). Optional quantization represents both weights and activations as signed 8-bit integers (W8A8). This path uses symmetric quantization, 32-bit integer accumulation, and fp32 rescaling, with Arm `dotprod` or x86 AVX-VNNI instructions for integer products. Weight scales are per output channel; activation scales are computed dynamically per token for linear layers and per input tensor for convolutions, using the maximum absolute value divided by 127. Selected operator groups use this path; normalization, nonlinearities, and unsupported output widths remain fp32. IMPACT’s cached text encoder remains fp32. Section VI-D evaluates these choices per target.

V. IMPACT

IMPACT (Instruction-Modulated Perception and Action Chunking with Transformers) adds language-based task selection to an action-chunking policy while keeping most language computation outside the repeated inference loop. Following ACT [5], it maps two camera views, robot state, and an instruction to a 50-action chunk (Fig. 3).

a) Visual and state inputs: The views share an ImageNet-initialized ResNet-18 [40]; batch normalization is folded into convolutions for inference. The SO-101 configuration uses 480×640 images, producing 600 visual tokens across both views, and predicts six-dimensional joint and gripper commands. The LIBERO configuration uses 256×256 images, producing 128 visual tokens, with an

eight-dimensional end-effector state and seven-dimensional relative actions. Linear projections map visual features and robot state to width 512.

b) Language conditioning: A frozen T5-small text encoder [41] maps the instruction to $E \in \mathbb{R}^{n_\ell \times 512}$, where $n_\ell \leq 32$ is the number of non-padding tokens. Longer inputs are truncated, and padding is masked within T5. These states condition the policy in two ways. First, projected text tokens are appended to the transformer encoder sequence, with padding excluded. Second, the mean text-encoder state, $\bar{e} = n_\ell^{-1} \sum_j E_j$, modulates the feature map F_i after the residual addition and ReLU of each backbone stage i :

$$[\gamma_i; \beta_i] = G_i \bar{e} + b_i, \quad F_i \leftarrow (1 + \gamma_i) \odot F_i + \beta_i, \quad (5)$$

Here, G_i and b_i are learned parameters, and the channel-wise scales γ_i and shifts β_i are broadcast over pixels; $\odot$ denotes element-wise multiplication. Initializing G_i and b_i to zero makes FiLM an identity operation at the start of training. The engine caches the projected text tokens and FiLM coefficients per instruction and applies the modulation to each query’s features.

c) Action prediction and training: A six-layer transformer encoder and four-layer decoder, with eight attention heads and feed-forward width 3200, predict the chunk from 50 learned action queries in one pass. The encoder receives the visual and text tokens, one state token, and one latent token. Visual tokens use two-dimensional sinusoidal positions; state, latent, and text positions are learned. Training follows ACT’s conditional variational autoencoder (CVAE) formulation: a four-layer encoder receives the demonstrated actions, state, and instruction and predicts a distribution over a 32-dimensional latent z . The objective combines L1 action reconstruction with a Kullback–Leibler penalty of weight 10 toward a standard normal latent prior; inference sets $z = 0$. State and actions are normalized by the training-set mean and standard deviation; predictions are converted back to robot units before execution. IMPACT has 60 M parameters in its per-query policy and 35.3 M in the frozen T5-small encoder, whose outputs are cached per episode.

d) Computational cost: For the SO-101 configuration, counting a multiply-add as two floating-point operations gives approximately 88 GFLOP per query. Text encoding, projection, and FiLM coefficient generation add about 1.2 GFLOP once per episode. Relative to the same six-layer encoder and four-layer decoder without language, up to 32 text tokens add about 2.3 GFLOP per query and FiLM adds 9 MFLOP, approximately 2.6% of the total operation count. Compared with the shallower ACT in Table II, IMPACT takes $1.20\text{--}1.33\times$ as long per query across the four CPUs.

VI. EXPERIMENTS

A. Setup

a) Hardware and policies: We evaluate six policies on four CPUs, including a passively cooled Pi 5 (Tables I and II). CPU benchmarks use checkpoints fine-tuned on 30 Hz SO-101 data, except for TurboVLA, whose LIBERO

TABLE I
CPU PLATFORMS, ENGINE THREAD COUNTS, AND
MATRIX-MULTIPLICATION KERNELS. P/E: PERFORMANCE/EFFICIENCY
CORES; C/T: CORES/HARDWARE THREADS.

Device	CPU core	ISA	Cores	Kernel	Thr.
Apple M4	Apple	NEON	4P+6E	Accelerate	8
Intel i9-14900HX	Raptor Lake	AVX2	8P+16E	SMK 6×16	16
AMD Ryzen 5 5500	Zen 3	AVX2	6C/12T	SMK 6×16	12
Raspberry Pi 5	Cortex-A76	NEON	4C	SMK 4×16	4

TABLE II
EVALUATED POLICIES AND AVAILABLE ACTION WINDOWS H . “+T5”
DENOTES A SEPARATE FROZEN TEXT ENCODER.

Policy	Params	Vision / language	Action model	H
ACT [5]	34 M	ResNet-18 / -	CVAE transformer	100
DP [6]	278 M	ResNet-18 / -	U-Net, 100/10 steps	32
Octo-Small [3]	27 M+T5	conv stem / T5-base	diffusion, 20 steps	4
TurboVLA [11]	0.2 B	DINOv3 / BERT	ACT-style decoder	12
SmolVLA [4]	450 M	SmolVLM2	flow matching	50
IMPACT (ours)	60 M+T5	ResNet-18 / T5-small	CVAE transformer	50

TABLE III
ACTION-SUPPLY RATE f_{eff} (ACTIONS/S). GRAY: FAILS C1; *italic*: MEETS
C1 ONLY; UPRIGHT BLACK: MEETS C2, AT $f_c = 30$ HZ. LAST TWO
COLUMNS: AFTER A 90 S PI 5 THERMAL SOAK.

Policy	H	Raspberry Pi 5							
		M4	i9	Ryzen	f_{eff}	T_q (s)	fp32	int8	
ACT	100	1150	892	627	109	0.92	89.9	242	
DP, 100 steps	32	7.2	7.2	3.6	0.8	40.1	-	-	
DP, 10 steps	32	58	55	29.3	6.3	5.08	-	-	
Octo-Small	4	76	48	48	7.8	0.51	5.8	11.3	
TurboVLA	12	87	58	57	9.3	1.28	7.2	-	
SmolVLA	50	73	42	38	6.1	8.19	-	-	
IMPACT	50	453	334	262	41	1.21	33.5	81.2	

checkpoint is evaluated against the same action-supply budget. Diffusion Policy uses one checkpoint with 100-step DDPM [42] or 10-step DDIM [43] sampling.

b) Measurement protocol: We time each query from observation preprocessing through conversion of predicted actions to robot units, as defined in Section IV. Nominal latency is the median of 15 queries after warmup, measured in a fresh process with the CPU initially at its idle temperature. To compare settings A and B, we interleave measurements in A–B–B–A order within one session to limit thermal drift. Before timing fp32 implementations, we check their outputs against the corresponding reference policy; quantization is evaluated separately.

B. Action Supply and Thermal Effects

We test whether each policy supplies enough actions for lagged execution (C1) or for execution that discards stale actions (C2). To assess the effect of heating, we also measure latency after 90 s of continuous inference on the passively cooled Pi 5.

Table III and Fig. 1 show that most evaluated policies meet C1 on desktop CPUs, while only ACT and IMPACT

qualify on the Pi 5. IMPACT is the only evaluated language-conditioned policy to supply at least 30 actions/s on the Pi 5. Octo-Small queries are shorter, but their four-action window is insufficient for either budget. After this workload, IMPACT supplies 33.5 actions/s in fp32 and 81.2 with int8, satisfying C1 and C2, respectively.

C. Speed and Numerical Fidelity

We compare `vla.simd` with compiled PyTorch using default-mode `torch.compile` [44] and the Inductor CPU backend. We also measure uncompiled (eager) execution, using PyTorch 2.11 for five policies and 2.13 for TurboVLA. All implementations use fp32 and the same query-timing boundary over six interleaved blocks after warmup; compilation time is excluded. Both implementations cache instruction-only encodings where supported and use the same thread count, with one PyTorch inter-op thread. We check numerical fidelity against each policy’s PyTorch reference using identical observations and stochastic inputs, with preset tolerances of 10^{-2} absolute error and 10^{-4} relative to action magnitude. Paired timings are collected in the same session and may differ from the nominal measurements in Table III.

For each configuration–CPU pair, speedup is the median within-block ratio $T_{\text{compile}}/T_{\text{engine}}$. The median across 28 pairs (six policies, with two DP samplers, on four CPUs) is approximately $1.4\times$, spanning $0.83\text{--}3.37\times$. Eager and compiled execution have similar latency in most settings; Octo-Small on the M4 shows the largest gain from compilation. On the Ryzen, Octo-Small and SmolVLA cross C1 (Fig. 4). The M4 regressions do not change C1 feasibility, although 10-step Diffusion Policy loses its C2 margin.

A separate numerical comparison on 2–12 observations per reported policy gives maximum absolute action differences of at most 8.8×10^{-5} in robot units. These small discrepancies are consistent with changes in floating-point accumulation order. The Intel and AMD backends produce bitwise-identical fp32 actions on the tested inputs. For IMPACT, the reference is our PyTorch implementation. Task performance is evaluated separately below.

D. Processor-Specific Implementation Choices

We vary eleven settings individually on ACT using the paired protocol above. We report a consistent improvement only when a setting is faster in all but at most one pair and its median effect exceeds baseline drift. In Table IV, bf16 expansion converts bfloat16 weights to fp32; the parallel cutoff is the minimum element count for parallel execution. The higher/lower thread counts are 10/4 on the M4, 32/8 on the i9, 12/6 on the Ryzen, and 4/2 on the Pi 5. We also compare W8A8 with fp32 on supported policy–CPU pairs and examine selective quantization of convolution and attention layers. The W8A8 comparisons quantize all supported operator groups: for IMPACT, these are ResNet convolutions, token projections, and encoder/decoder linear layers, with the text encoder kept in fp32. Varying one setting at a time measures its individual effect; interactions between settings are not evaluated.

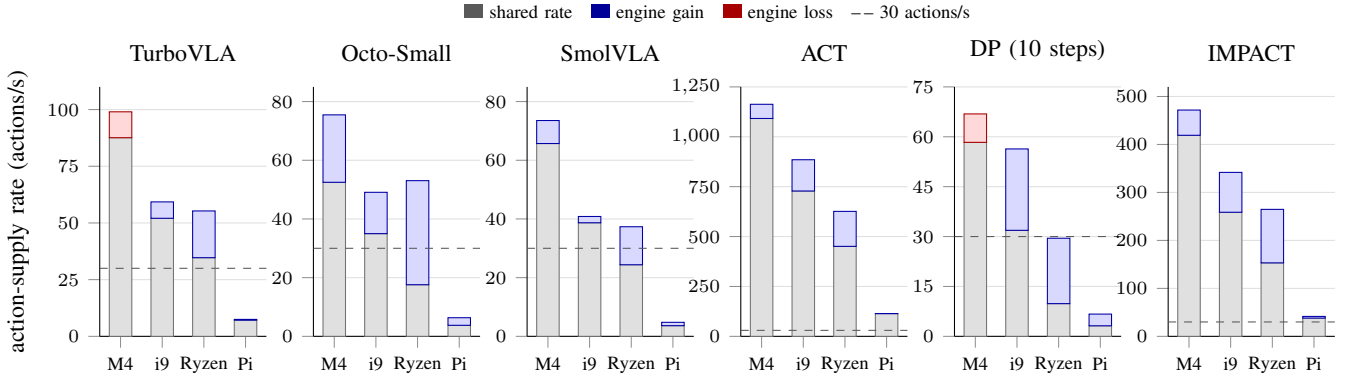


Fig. 4. CPU action-supply rates for six policy configurations on four CPUs; panel scales differ. Blue segments show the rate gained by `vla.simd` over the `torch.compile` baseline, and red segments show a loss. The dashed line marks 30 actions/s; it does not denote feedback frequency. On the Ryzen, Octo-Small and SmolVLA cross this line with `vla.simd`. Diffusion Policy with 100 steps is omitted from the figure.

TABLE IV

EFFECTS OF INDIVIDUAL RUNTIME SETTINGS ON ACT. A/B:

CONSISTENTLY FASTER SETTING; =: NO CONSISTENT DIFFERENCE.

VALUES ARE MEDIAN $100(T_B/T_A - 1)$ (%). BOTTOM: W8A8 SPEEDUP OVER FP32 (×); N/A: UNSUPPORTED; -: UNMEASURED.

Setting	A/B	M4	i9	Ryzen	Pi 5
Attention kernel	library / custom	A +9.9	=	=	=
bf16 expansion	off / on	=	=	=	=
Min. panel rows	16 / 4	=	=	B -4.2	=
Convolution	tiled / flat	=	A +156	A +260	A +16.9
Block rows	auto / 64	=	A +8.3	A +5.0	=
Thread schedule	static / dyn.	=	=	=	=
Parallel cutoff	8192 / 1024	=	=	=	=
Weight packing	on / off	=	A +31.0	A +30.7	A +14.8
Thread count	more / fewer	=	A +5.4	=	A +32.0
Threads per view	auto / 2	B -6.2	A +104	A +75.6	A +17.8
Thread waiting	passive / active	A +72.9	A +1.7	=	=
W8A8, ACT		1.13	1.24	n/a	2.68
W8A8, IMPACT		1.06	1.26	n/a	2.39
W8A8, Octo-Small		0.66	1.03	n/a	1.73
W8A8, SmolVLA		0.71	1.14	n/a	-

Table IV shows that useful settings depend on the target CPU: tiled convolution benefits the non-Apple backends, while extra camera-view threads help the M4 but hurt the other devices. Quantization is similarly uneven, yielding substantial gains on the Pi 5 but slowing some policies on the M4. For IMPACT on the M4, quantizing only convolutions is faster than quantizing all layers. These results support selecting settings and quantized layers through measurements on each target.

E. Manipulation and Instruction Following

a) *LIBERO*: We evaluate IMPACT on four LIBERO simulation suites [45], each containing ten manipulation tasks. We train one policy per suite without robot pretraining and evaluate each task from 20 initial states on a GPU host. Training uses AdamW with batch size 64, seed 1000, constant learning rate 10^{-4} , and weight decay 10^{-4} . Evaluation uses two 256×256 views and executes all 50 relative

TABLE V

LIBERO EVALUATION ON THE FOUR MANIPULATION SUITES (%).

Policy	Spatial	Object	Goal	Long	Avg
<i>No robot pretraining</i>					
Diffusion Policy	78.3	92.5	68.3	50.5	72.4
TurboVLA	99.2	99.8	97.4	94.2	97.7
IMPACT, final	83.5	83.5	84.5	54.0	76.4
IMPACT, selected	83.5	83.5	89.0	58.5	78.6
<i>Pretrained on robot data</i>					
Octo-Base	78.9	85.7	84.6	51.1	75.1
OpenVLA	84.7	88.4	79.2	53.7	76.5
SmolVLA	90.0	96.0	92.0	71.0	87.3

end-effector actions before querying the policy again. The final 30,000-step checkpoints achieve 76.4% mean success (Table V). Selecting checkpoints on these same evaluation rollouts yields 78.6%; we report this exploratory result separately because selection can inflate the estimate. Published DP, Octo-Base, and OpenVLA results average three seeds of 500 rollouts per suite; TurboVLA uses 500 and SmolVLA 100 [46], [11], [4]. These published results provide context rather than a controlled comparison.

b) *Instruction following*: To test whether IMPACT uses language to select a task, we keep the LIBERO-Goal environments fixed and change only the instruction, using the selected 25,000-step checkpoint. All ten tasks share the same scene and objects. We evaluate three instruction permutations, each replacing every original instruction, for 600 rollouts in total. We score both the original and newly instructed goals using LIBERO’s success conditions. IMPACT achieves the newly instructed goal in 91% of trials and the original goal in none, against an 88.5% correct-instruction control. This intervention supports language-based selection among familiar goals; generalization to unseen wording and the individual contributions of the two conditioning paths remain untested.

c) *SO-101*: On the SO-101 arm (Fig. 5), we use 44 demonstrations for three instructions in a shared scene and 20 for a separate drawer task. LeRobot’s asynchronous

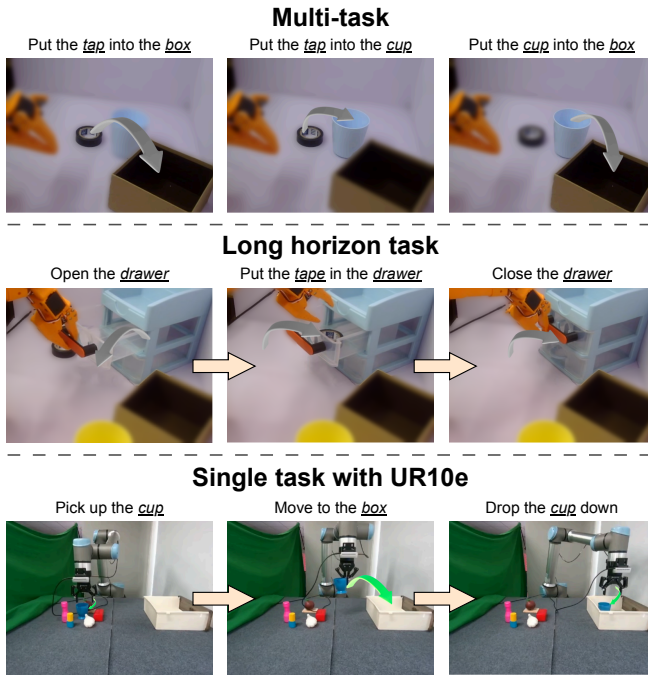


Fig. 5. Real-robot tasks on two embodiments. Top: three instructions in a shared SO-101 [37] scene vary the destination or grasp target. Middle: the SO-101 opens a drawer, places the tape inside, and closes it. Bottom: a UR10e with a Robotiq gripper picks up the cup and places it in the box. The middle and bottom rows show successive stages from left to right.

interface [21] connects to `vla.simd` on the M4, Pi 5, and i5-12400F, or to PyTorch on a GPU. The trials use one scene and one operator; IMPACT results pool trials across these CPU and GPU servers. Each row of Table VI reports 20 trials with a Wilson 95% confidence interval. IMPACT performs better on destination selection than on changing the grasp target (Table VI). The fp32 and int8 success counts are similar, although 20 trials per condition are insufficient to establish equivalent performance.

As a no-language control, we train ACT with IMPACT’s chunk length, transformer depth, and shared ResNet-18, removing text and FiLM. This control is deeper than the ACT in the CPU benchmarks. We train one policy on all three tasks and three specialist policies on one task each. All train for 19,000 steps with batch size 8 and execute synchronously, so inference and motion do not overlap. Each specialist therefore receives more passes over its smaller dataset. The multi-task control succeeds in 4 of 20 trials, compared with 46 of 60 across specialists. The operator attributes 9 of the 20 multi-task trials to target ambiguity and 7 to grasp failures. These observations suggest task ambiguity as a failure mode, but differences in data exposure and execution protocol prevent attributing the success gap to language conditioning alone.

d) *UR10e*: SmolVLA, served by `vla.simd` from the M4 and Ryzen, places a cup into a box using a UR10e with a Robotiq gripper (Table VII). Both hosts achieve 12 successes in 20 trials, with a $2.0\times$ difference in mean query round-trip time. The small samples leave the effect of host latency unre-

TABLE VI
SO-101 ROLLOUT RESULTS.

Model	Goal	Training	SR (% , 95% CI)
ACT	all three	multi-task	20 (8–42)
ACT	tape $\rightarrow$ box	single-task	60 (39–78)
ACT	tape $\rightarrow$ cup	single-task	90 (70–97)
ACT	cup $\rightarrow$ box	single-task	80 (58–92)
IMPACT	tape $\rightarrow$ box	multi-task	90 (70–97)
IMPACT	tape $\rightarrow$ cup	multi-task	85 (64–95)
IMPACT	cup $\rightarrow$ box	multi-task	60 (39–78)
IMPACT int8	tape $\rightarrow$ box	multi-task	85 (64–95)
IMPACT int8	tape $\rightarrow$ cup	multi-task	95 (76–99)
IMPACT int8	cup $\rightarrow$ box	multi-task	50 (30–70)
IMPACT	tape $\rightarrow$ drawer	drawer	65 (43–82)

TABLE VII
UR10E ROLLOUT RESULTS.

Host	Round-trip (ms)	SR (% , 95% CI)
Apple M4	1177	60 (39–78)
Ryzen 5 5500	2343	60 (39–78)

solved. Round-trip times include transport and serialization, exceeding nominal engine latency by approximately 0.5 s on the M4 and 1.0 s on the Ryzen. These additional delays show why deployment must be assessed using the full observation-to-action path.

VII. CONCLUSION

We presented `vla.simd`, an inference engine that combines shared SIMD kernels with target-specific optimization, and IMPACT, an ACT-based policy that adds language-guided task selection with little repeated computation. CPU deployment becomes feasible when the execution horizon covers inference delay and the implementation uses the processor efficiently. On the Ryzen, `vla.simd` pushes Octo-Small and SmolVLA across the 30Hz threshold. On the Pi 5, IMPACT is the only language-conditioned policy in our evaluated set that meets this budget, and its int8 path also meets the stricter time-aligned budget after a thermal soak. Instruction shuffling and real-robot trials demonstrate language-based goal selection, extending ACT to instruction-guided manipulation.

VIII. LIMITATIONS

Our evaluation covers policies up to 450 M parameters and compares CPU inference with eager and compiled PyTorch; it does not benchmark GPUs, embedded accelerators, or dedicated runtimes such as ONNX Runtime and OpenVINO. The budgets use median engine latency, whereas uninterrupted deployment depends on the tail of the full observation-to-action delay. Thermal behavior is measured over one 90 s soak. The int8 path is validated on the SO-101 multi-task bench, but its effect on simulation success and on the long-horizon task is unmeasured. LIBERO evaluation uses one seed, and the SO-101 trials pool servers, limiting conclusions about behavior on individual CPUs.

ACKNOWLEDGMENTS

Disclosure of AI tools. Generative AI assisted this work strictly in these three roles—literature, writing, and coding—and in no other. AI tools suggested literature candidates, which the authors checked against original sources. OpenAI Codex and Anthropic Claude Code assisted prose throughout the paper, the $\text{\LaTeX}$ source of Fig. 4, and portions of training and evaluation scripts. The authors reviewed the text, tested the code against reference outputs, and executed all experiments; AI generated no experimental results. The research design and interpretation are the authors’ own, and they take responsibility for the manuscript.

REFERENCES

- [1] M. J. Kim *et al.*, “OpenVLA: An open-source vision-language-action model,” in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 270, 2024, pp. 2679–2713.
- [2] K. Black *et al.*, “ π_0 : A vision-language-action flow model for general robot control,” in *Robotics: Science and Systems (RSS)*, 2025.
- [3] Octo Model Team *et al.*, “Octo: An open-source generalist robot policy,” in *Robotics: Science and Systems (RSS)*, 2024.
- [4] M. Shukor *et al.*, “SmolVLA: A vision-language-action model for affordable and efficient robotics,” *arXiv preprint arXiv:2506.01844*, 2025.
- [5] T. Z. Zhao *et al.*, “Learning fine-grained bimanual manipulation with low-cost hardware,” in *Robotics: Science and Systems (RSS)*, 2023.
- [6] C. Chi *et al.*, “Diffusion policy: Visuomotor policy learning via action diffusion,” in *Robotics: Science and Systems (RSS)*, 2023.
- [7] K. Black, M. Y. Galliker, and S. Levine, “Real-time execution of action chunking flow policies,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025, pp. 33 383–33 407.
- [8] K. Goto and R. A. van de Geijn, “Anatomy of high-performance matrix multiplication,” *ACM Transactions on Mathematical Software*, vol. 34, no. 3, pp. 1–25, 2008.
- [9] F. G. Van Zee and R. A. van de Geijn, “BLIS: A framework for rapidly instantiating BLAS functionality,” *ACM Transactions on Mathematical Software*, vol. 41, no. 3, pp. 1–33, 2015.
- [10] K. Black *et al.*, “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” *arXiv preprint arXiv:2504.16054*, 2025.
- [11] H. Xie *et al.*, “TurboVLA: Real-time vision-language-action model at 32 Hz on an RTX 4090 with <1 GB VRAM,” *arXiv preprint arXiv:2607.27205*, 2026.
- [12] Y. Yang *et al.*, “EfficientVLA: Training-free acceleration and compression for vision-language-action models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025.
- [13] J. Wen *et al.*, “TinyVLA: Toward fast, data-efficient vision-language-action models for robotic manipulation,” *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3988–3995, 2025.
- [14] J. Chen *et al.*, “NanoVLA: Routing decoupled vision-language understanding for nano-sized generalist robotic policies,” *arXiv preprint arXiv:2510.25122*, 2025.
- [15] H. Wang *et al.*, “BitVLA: 1-bit vision-language-action models for robotics manipulation,” *arXiv preprint arXiv:2506.07530*, 2025.
- [16] Z. Yu *et al.*, “A survey on efficient vision-language-action models,” *arXiv preprint arXiv:2510.24795*, 2025.
- [17] W. Jiang *et al.*, “How fast can I run my VLA? demystifying VLA inference performance with VLA-Perf,” *arXiv preprint arXiv:2602.18397*, 2026.
- [18] M. Vishwanathan, S. Subramanian, and A. Raghunathan, “Characterizing VLA models: Identifying the action generation bottleneck for edge AI architectures,” *arXiv preprint arXiv:2603.02271*, 2026.
- [19] K. Zhou *et al.*, “Characterizing vision-language-action models across XPU: Constraints and acceleration for on-robot deployment,” *arXiv preprint arXiv:2604.24447*, 2026.
- [20] J. Williams *et al.*, “Lite VLA: Efficient vision-language-action control on CPU-bound edge robots,” *arXiv preprint arXiv:2511.05642*, 2025.
- [21] R. Cadene *et al.*, “LeRobot: An open-source library for end-to-end robot learning,” in *International Conference on Learning Representations (ICLR)*, 2026.
- [22] Y. Liu *et al.*, “Bidirectional decoding: Improving action chunking via guided test-time sampling,” in *International Conference on Learning Representations (ICLR)*, 2025.
- [23] E. Perez *et al.*, “FiLM: Visual reasoning with a general conditioning layer,” in *AAAI Conference on Artificial Intelligence*, 2018.
- [24] E. Jang *et al.*, “BC-Z: Zero-shot task generalization with robotic imitation learning,” in *Conference on Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research, vol. 164, 2021, pp. 991–1002.
- [25] A. Brohan *et al.*, “RT-1: Robotics transformer for real-world control at scale,” in *Robotics: Science and Systems (RSS)*, 2023.
- [26] S. Haldar, Z. Peng, and L. Pinto, “BAKU: An efficient transformer for multi-task policy learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024, pp. 141 208–141 239.
- [27] H. Bharadhwaj *et al.*, “RoboAgent: Generalization and efficiency in robot manipulation via semantic augmentations and action chunking,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 4788–4795.
- [28] T. M. Low *et al.*, “Analytical modeling is enough for high-performance BLIS,” *ACM Transactions on Mathematical Software*, vol. 43, no. 2, pp. 1–18, 2016.
- [29] Google, “XNNPACK: High-efficiency floating-point neural network inference operators for mobile, server, and web,” <https://github.com/google/XNNPACK>, 2026.
- [30] ONNX Runtime developers, “ONNX Runtime: Cross-platform, high performance ML inferencing and training accelerator,” <https://github.com/microsoft/onnxruntime>, 2026.
- [31] Intel Corporation, “OpenVINO toolkit,” <https://github.com/openvinotoolkit/openvino>, 2026.
- [32] M. Nachin *et al.*, “ExecuTorch – a unified PyTorch solution to run AI models on-device,” *arXiv preprint arXiv:2605.08195*, 2026.
- [33] G. Gerganov *et al.*, “llama.cpp,” <https://github.com/ggml-org/llama.cpp>, 2023.
- [34] K. D. Nguyen *et al.*, “vla.cpp: A unified inference runtime for vision-language-action models,” *arXiv preprint arXiv:2606.08094*, 2026.
- [35] W. Thomason, Z. Kingston, and L. E. Kavraki, “Motions in microseconds via vectorized sampling-based planning,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 8749–8756.
- [36] C. W. Ramsey *et al.*, “Collision-affording point trees: SIMD-amenable nearest neighbors for fast collision checking,” in *Robotics: Science and Systems (RSS)*, 2024.
- [37] R. Knight *et al.*, “Standard open SO-100 & SO-101 arms,” <https://github.com/TheRobotStudio/SO-ARM100>, 2024.
- [38] Apple Inc., “Accelerate,” <https://developer.apple.com/documentation/accelerate>, 2026.
- [39] K. Chellapilla, S. Puri, and P. Simard, “High performance convolutional neural networks for document processing,” in *International Workshop on Frontiers in Handwriting Recognition (IWFHR)*, La Baule, France, 2006. [Online]. Available: <https://inria.hal.science/inria-00112631>
- [40] K. He *et al.*, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [41] C. Raffel *et al.*, “Exploring the limits of transfer learning with a unified text-to-text transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [42] J. Ho, A. Jain, and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 6840–6851.
- [43] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [44] J. Ansel *et al.*, “PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation,” in *ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2024, pp. 929–947.
- [45] B. Liu *et al.*, “LIBERO: Benchmarking knowledge transfer for lifelong robot learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [46] M. J. Kim, C. Finn, and P. Liang, “Fine-tuning vision-language-action models: Optimizing speed and success,” in *Robotics: Science and Systems (RSS)*, 2025.